

Incident Postmortem: Spring4Shell Malware Attack

Incident ID: Telstra Cybersecurity Simulation – Spring4Shell Exploit

Date March 20, 2022

Duration: About 2 hours before it was under control

Severity: P1 - Critical

Summary

At 03:16:34 UTC, The logs showed signs of a malware attack on the NBN service. The exploit was the Spring4Shell vulnerability (CVE-2022-22965). The attacker tried to drop a malicious JSP file (`tomcatwar.jsp`) that could run commands on the server.

Service was impacted until we pushed out a firewall rule to block the exploit traffic. Everything was stable again about two hours later.

Timeline

- 03:15 UTC – Logs showed suspicious traffic and alerts spiked.
- 03:20 UTC – Confirmed NBN was affected.
- 03:25 UTC – Sent out the incident notification email to the NBN team.
- 03:40 UTC – Verified the exploit payload in the traffic.
- 04:00 UTC – Escalated to the Networks team for firewall mitigation.
- 05:10 UTC – Firewall rule went live blocking the bad path and headers.
- 05:20 UTC – Malicious traffic dropped off.
- 05:30 UTC – Systems were stable.

Impact

- The NBN service (critical infrastructure) experienced downtime and impaired functionality.
- Attempted remote code execution (RCE) could have allowed full system compromise.
- Customer-facing services relying on NBN were at risk during the two-hour window.

Detection

- Initial alerts came from firewall logs showing repeated malicious requests.
- Logs showed repeated malicious requests containing known exploit headers (`suffix`, `c1`, `c2`, etc.) and targeting `/tomcatwar.jsp`.
- Confirmed the exploit was Spring4Shell (CVE-2022-22965).
- Analysis of the payload showed attempts to execute system commands via JSP webshell.

Incident Postmortem: Spring4Shell Malware Attack

Root Cause

- Vulnerability: Spring4Shell (CVE-2022-22965).
- Vector: Malicious HTTP request with bad headers (`suffix`, `c1`, `c2`, etc.) and the `/tomcatwar.jsp` path.
- Impact: If successful, it would've given the attacker remote code execution.

Resolution

The incident was resolved by deploying a targeted firewall rule to block malicious requests tied to the Spring4Shell exploit. The rule filtered traffic aimed at `/tomcatwar.jsp` and requests carrying the exploit-specific headers. Once the rule was live, the attack traffic was successfully blocked and service stability was restored. Additional monitoring confirmed no further attempts were successful.

Action Items

- Triage
- Alert
- Firewall
- Validate

Lessons Learned

Went well:

- Quick triage and escalation.
- Teams communicated well.
- Firewall rule worked and stopped the attack.

Needs Improvement:

- Should've patched Spring earlier.
- Would help to have automated alerts for exploit payload patterns.
- A clear runbook for zero-days would save time.

Next Steps

1. Patch Spring Framework to 5.3.18+.
2. Add detection rules for this payload to monitoring.
3. Build more automation into SOC playbooks.
4. Share this postmortem with the team for training.
5. Keep the logs, emails, and rule as part of compliance records.